
On End-to-End Test-Time-Training for 3D Reconstruction

Hongyu Ma¹

¹University of Maryland

Abstract

VGGT showed end-to-end feedforward approaches to 3D reconstruction can match or exceed classical visual 3D reconstruction methods while operating at a fraction of the computational cost. However, Transformer-based architectures and classical methods suffer alike from a quadratic asymptotic time complexity with respect to sequence length. This paper proposes Per-Pixel 3D Reconstruction (PP3DR), an end-to-end Test-Time-Training architecture for 3D reconstruction. Through various architectural improvements and reformulations, this paper aims to show that an end-to-end TTT architecture for 3D reconstruction can be trained from scratch to match current state-of-the-art approaches relying on quadratic Softmax attention. Code is available at <https://github.com/hughmathimati/PP3DR>.

1 Introduction

3D reconstruction from casual monocular video is the field of taking as input a sequence of RGB video frames and outputting the mapped 3D coordinates of each pixel in each frame, alongside the camera poses within those coordinates from which each frame was captured. For many years since its debut, it was dominated by classical Structure-from-Motion (SfM) pipelines ([6, 16, 19]), the latest of which relied on graph-based optimisation.

Starting with VGGT [23], end-to-end feedforward approaches to 3D reconstruction have been shown to produce results on-par with classical methods while operating at a fraction of the computational cost. However, Transformer-based architectures and classical methods suffer alike from a quadratic asymptotic time complexity with respect to sequence length; Transformers rely on Softmax attention, while classical graph-based optimisation must optimise across $\mathcal{O}(N^2)$ graph edges (between frames). With that said, it is important to keep in mind that TTT [21] is not a panacea for long sequence lengths; one could argue that the speed-quality trade-off now happens at the fast-weight and inner-loop gradient step level. This is an unavoidable consequence of choosing a fixed state size; the bottleneck happens either at the information level or the computation level.

The proposed architecture, Per-Pixel 3D Reconstruction (PP3DR), is an end-to-end Test-Time-Training architecture for 3D reconstruction from casual monocular videos. Thus, it runs with a linear time complexity both with respect to frame-sequence length as well as with respect to the resolution of the input image. Aside from paving the way for high-quality 3D reconstruction to run on edge devices, the lower computation cost associated with the linear time complexity of attention both with respect to image resolution and sequence length provides two main benefits:

1. The linear time-complexity of TTT with respect to image resolution allows inference to be performed at or near the original image resolution, rather than downscaling for evaluation and upscaling before computing the metrics, potentially leading to higher-quality results.
2. The linear time-complexity of TTT with respect to sequence length allows very long sequences (hundreds of frames or more) to be processed in a single linear-time pass, whereas

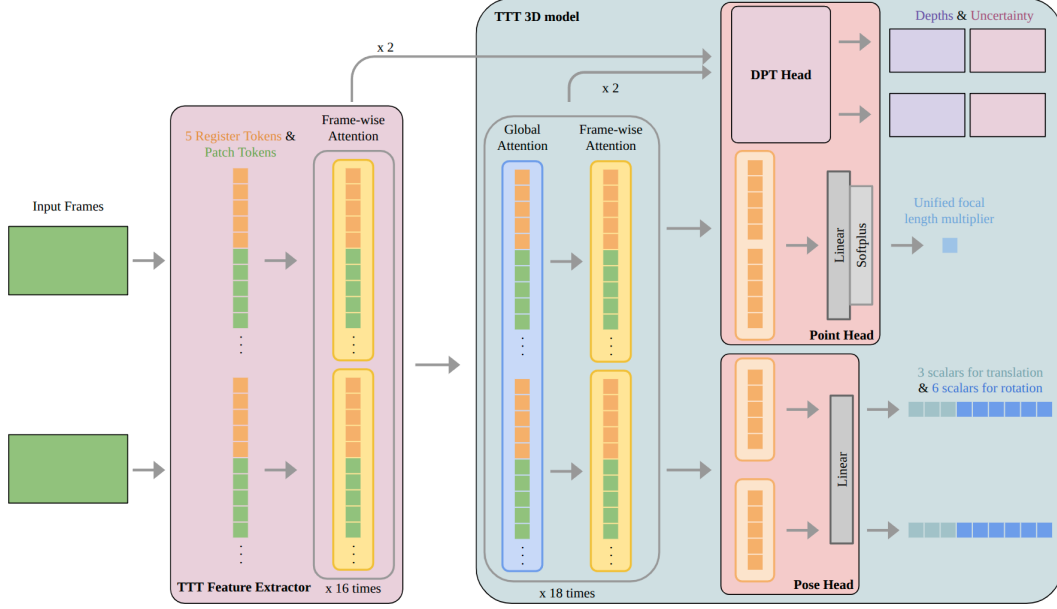


Figure 1: Overview of the full PP3DR architecture.

traditional methods relying on quadratic attention would require splitting the sequence into shorter chunks before geometrically concatenating the intermediate results.

In addition to substituting TTT for softmax attention, PP3DR adopts additional architectural modifications:

- Like Pi3 [26], PP3DR predicts relative camera poses from the current frame to the next, although with a slightly different parameterisation.
- While VGGT predicts quaternion rotations and Pi3 uses SVD to diagonalise a 3x3 predicted rotation matrix, PP3DR uses the 6D rotation parameterisation publicised in [29].
- Like VGGT and Pi3, PP3DR uses a depth-weighted point loss, although with a different weighting formula.
- Like VGGT and DepthAnything3 [10], PP3DR predicts log depth with aleatoric uncertainty, but its aleatoric uncertainty is not capped at a minimum of 1.
- PP3DR’s depth head is similar to VGGT-Omega’s [24] depth head but uses end-to-end upscaling, with residual transpose convolutions for added detail (in addition to other changes mentioned in the Experiments section).

2 Related Work

The 3D reconstruction scene was revolutionised by the seminal VGGT paper [23], which showed that feedforward end-to-end learned deep neural networks could match the performance of classical-SLAM-based approaches to dense point-cloud reconstruction from casual monocular videos at a fraction of the compute time. Since then, many variations have been proposed to the original approach of [23]; Wang et al. [26] propose a permutation-equivariant loss formulation, and Lin et al. propose Depth Anything 3 [10], which surpasses VGGT in reconstruction and pose-estimation quality. Most recently, Wang et al. released a larger and updated version of VGGT called VGGT- ω [24], which holds the current SoTA in 3D reconstruction.

Separately, Sun et al. [21] originally proposed the Test-Time Training formulation as an alternative to quadratic Softmax attention with linear time complexity and promising results. However, it incurred a debilitating constant factor it due to its naive recurrent approach, leading Tianyuan Zhan et. al. to propose LaCT [28], an architecture which makes improvements to both the architecture and the

proposed approach of [21]. It has been tested in real-world scenarios, most notably ViTTT³ [5], where Hna et al. demonstrate a feature extractor based entirely on TTT blocks.

Test Time Training has seen active exploration since the debut of Sun et al.’s foundational paper, but Liu et al. [12] found, among many other things, that maintaining only the final SwiGLU weight layer as the TTT block fast weights (reverting the other two blocks to slow weights) provides slightly better performance across the board. Thus, the TTT blocks used in PP3DR use their implementation instead of the traditional LaCT implementation.

Test-time-training in 3D reconstruction is not a new idea. Zhang et al. [27] leverage TTT to propose a sliding-window approach for very long video sequences; Chen et al. [3] propose a TTT formulation of CUT3R [25] (with a persistent hidden state), and VGGT³ [4] finetunes VGGT with its Softmax attention replaced with TTT blocks.

In summary, both major components of a 3D reconstruction architecture (the feature extractor and the main 3D encoder blocks) have been individually experimented with in the TTT space. However, an end-to-end implementation, least of all one trained from scratch, has never been successfully demonstrated.

3 Methodology

At its core, PP3DR maps N input images $I_1, \dots, I_N \in \mathbb{R}^{3 \times H \times W}$ to N corresponding depth maps $D_1, \dots, D_N \in \mathbb{R}^{H \times W}$, $N - 1$ corresponding *relative* camera pose rotations $R_1, \dots, R_{N-1} \in \mathbb{R}^{3 \times 3}$ and translations $T_1, \dots, T_{N-1} \in \mathbb{R}^3$, and one unified focal length $f \in \mathbb{R}$. The principal point is assumed to be at the center of the image.

The camera pose is parameterised as the relative translation from the current frame to the next frame. Thus, for a sequence of L frames, it outputs $L - 1$ relative translations and $L - 1$ relative rotations. Let the absolute camera extrinsic for a given frame be:

$$E_i = [R_i | t_i]$$

Then, the relative translation is parameterised as $t_{i+1} - t_i$, and the relative rotation is parameterised as $R_{i+1}R_i^\top$.

3.1 PP3DR Architecture

PP3DR consists of a 16-block TTT feature extractor (modeled after DINOv3 [18]), followed by 18 blocks each of alternating global and local TTT blocks (modeled after VGGT [23]), for a total of 52 sequential blocks in the pipeline. A 2D (unnormalised) RoPE [20] is used for local attention (x and y-coordinates of the feature), and a 3D (unnormalised) RoPE is used for global attention (x and y-coordinates of the feature + frame index). It uses a simple linear projection from register tokens to decode the camera poses, and uses a custom DPT [14] head for (log) depth and (log) aleatoric depth-uncertainty prediction (see Fig. 1).

Each input image is resized so its longest dimension is a multiple of the patch size, and then padded along the other dimension to the nearest multiple of the patch size. The ground-truth camera intrinsics are adjusted accordingly, and the ground-truth (adjusted) principal point is used to unproject the predicted depths (i.e. it is not predicted by the model).

The model has two task-specific decoder heads: the pose head and the point head. The pose head flattens the register tokens in each frame to predict nine scalars per frame:

1. Three scalars for the (x, y, z) relative pose translation.
2. Six scalars for the 6D rotation parameterisation for relative pose rotation [29]. This is notably different from VGGT, which uses a quaternion parameterisation, and Pi3, which predicts nine scalars for an entire rotation matrix and then performs an SVD diagonalisation.

The point head predicts a single focal length per sequence, a dense log-depth map per frame, and a dense aleatoric depth uncertainty map per frame. The focal length is predicted with a linear projection on the register tokens, while the dense maps are predicted with a modified DPT head. The DPT head extracts two feature maps from the TTT feature extractor and two feature maps from the main PP3DR

3D encoder blocks. A residual transposed convolution is added onto the usual bilinear upsampling before the residual convolution at each upscale step for additional detail.

3.2 Aleatoric Uncertainty

All dense losses are additionally weighted by the predicted aleatoric uncertainty. For example, for a raw (depth-weighted) per-pixel depth loss of E , the aleatoric-weighted loss for that pixel is:

$$E' = \frac{E}{\sigma} + \log(\sigma)$$

... where $\sigma \in (0, \infty)$ is the predicted aleatoric uncertainty.

Following other SOTA models, PP3DR actually predicts *log* aleatoric uncertainty, meaning its aleatoric weighted uncertainty is instead calculated as:

$$E' = E \cdot e^{-s} + s$$

... where $s = \log \sigma \in (-\infty, \infty)$ is the predicted *log* aleatoric uncertainty.

Other SOTA models like DepthAnything3 and VGGT-Omega parameterise their aleatoric uncertainty as $e^s + 1$, such that $\sigma \in (1, \infty)$. However, the author found that this leads to much of the predicted aleatoric uncertainty map being pushed as small as possible, losing much detail in the process (see Fig. 7a). This is because the aleatoric-weighted loss is minimised when $s = E$, meaning when $E < 1$, as is relatively common in a log-depth parameterisation, the confidence map pushes its logits as far negative as possible, trying to achieve a minimum it can never reach.

It should be noted that removing the $+1$ can lead to negative aleatoric-weighted losses. This doesn't matter, however, because the optimiser only cares about the gradient of the loss, which still points in the right direction.

3.3 DPT Head

My resulting DPT head resembles VGGT-Omega's, but is modified in the following ways:

- VGGT-Omega takes all four of its feature maps from its main 3D encoder blocks (none of them from its feature extractor). PP3DR instead takes two of its feature maps from the feature extractor and two of its feature maps from the main 3D encoder blocks.
- VGGT-Omega's depth head scales its four feature maps to 4x, 2x, 1x, and 0.5x, respectively. PP3DR instead scales its feature maps to 8x, 4x, 2x, and 1x, with the goal of avoiding the potential information bottleneck involved in downscaling and then upscaling the fourth feature map.
- VGGT-Omega projects all its feature maps into 256 channels. PP3DR projects its 8x feature map into 128 channels instead. Importantly, bottlenecking channels late in the depth-head pipeline can lead to insufficient representability during the upscale stage, leading to suboptimal predictions; however, the author found that 128 channels at an 8x upscale (0.5x the original resolution) works fine.
- VGGT-Omega's upscale involves a bilinear interpolation followed by a residual convolution block. My depth head adds a transpose convolution to the bilinear interpolation output, with the intention of adding the capability to represent discontinuities which would be difficult to mimic with a bilinear interpolation alone. Along with the fact that each of its feature maps are at twice the resolution of their VGGT-Omega counterpart, this leads to a non-negligible increase in VRAM usage, and thus to the decision to decrease the number of channels of the 8x feature map from 256 to 128.
- VGGT-Omega uses a linear projection and pixel shuffle for the final 4x upscale. Similarly, PP3DR performs its final 2x upscale as a linear projection and pixel shuffle; the author found that the normal loss is quite effective at smoothing out patch artifacts.

VGGT-Omega opted to train its entire pipeline, including its Dinov3 feature extractor, from scratch. While the author had previously experimented with freezing a pre-trained TTT feature extractor backbone, it empirically led to blurry depth maps, even after fine-tuning with an unfrozen feature

extractor. Training the entire pipeline from scratch gave much more detailed depth maps; the author infers that this is due to the other layers in the feature extractor preventing any particular layer from changing in any significant manner, as the “purpose” of each block in a pre-trained feature extractor is already “locked in”.

Design decisions are further elaborated upon in Sec. 4.

4 Experiments

4.1 Datasets

Due to the limited availability of storage, dataset selection was largely limited to what was already available on the shared compute cluster. Given the initial proof-of-concept nature of this project, test and validation splits alike were trained on, with Sintel [2] being the sole true validation dataset. The train mix consisted of a mix of indoor datasets (Dynamic Replica [8], InteriorNet [9], NRGBD [1]), outdoor datasets (FlyingThings3D [13], RTMV [22], ETH3D [17]), and object-centric datasets (DTU [7]). DTU and ETH3D are real-world datasets, while all the other datasets mentioned are synthetic. The total combined train set contains on the order of 5,120 scenes of varying resolutions (from 480p up to 8k) and varying sequence lengths (from 5 up to over 1,000).

4.2 Training

Ground-truth depths and camera translations are normalised so the median depth of pixels across all frames is 10. The loss consists of:

1. A smooth L1 ($\beta = 0.01$) depth-weighted point loss on the 3D coordinates of each pixel unprojected into the camera’s coordinate system. Given a median ground-truth depth of Z , the weight of a point with ground-truth depth z_i is $\frac{2Z}{Z+z_i}$. For a median ground-truth depth of 1, this assigns the median point a weight of 1, closer points weights approaching 2, and farther points weights approaching zero. This weight was chosen because:

$$\frac{2}{1+Z} = \frac{2}{1+e^{\log(Z)}}$$

In other words, the weight is essentially twice a reverse sigmoid in log space. Each pixel’s point loss is additionally weighted by its predicted aleatoric uncertainty; thus, the full expression for the point loss is:

$$\sum_{i \in HW} \text{smooth_l1}([\hat{x}_i, \hat{y}_i, \hat{z}_i], [x_i, y_i, z_i], \beta = 0.01) \cdot \frac{2Z}{Z+z_i} \cdot e^{-\hat{s}_i} + \hat{s}_i$$

where the sum is over all pixels, $\hat{x}_i, \hat{y}_i, \hat{z}_i$ are the predicted (unprojected) camera coordinates of the point corresponding to pixel i , $\hat{s}_i$ is the predicted log aleatoric uncertainty for pixel i , x_i, y_i, z_i are the ground-truth camera coordinates of the point corresponding to pixel i , and Z is the median ground-truth depth.

2. A smooth L1 ($\beta = 0.01$) loss on log depth, weighted by the predicted aleatoric uncertainty.
3. A gradient-matching loss, calculated as a smooth L1 ($\beta = 0.01$) loss on the horizontal and vertical differences in log depth between adjacent pixels. Each gradient’s loss is weighted by the maximum aleatoric uncertainty of the two pixels comprising the difference.
4. A normal loss, calculated as a smooth L1 ($\beta = 0.01$) loss on the difference in radians between the ground-truth surface normal and the predicted surface normal. Each normal is weighted by the maximum aleatoric uncertainty of the neighboring pixels used to calculate the surface normal (via cross-product). Depth discontinuities are excluded from the normal loss.
5. A smooth L1 ($\beta = 0.01$) loss on relative camera pose translations, where the relative camera pose translation is as defined above.
6. A smooth L1 ($\beta = 0.01$) loss on the relative camera pose rotation angle (similar to in the normal loss), where the relative camera pose rotation is as defined above.

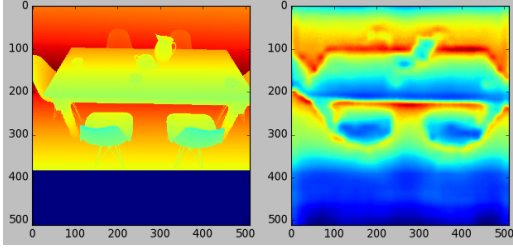


Figure 2: Final feature map only.

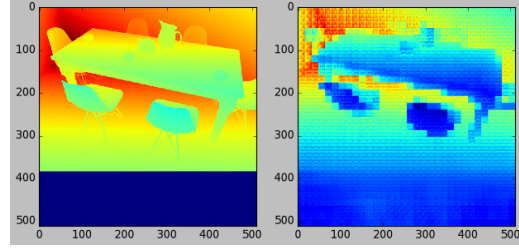


Figure 3: Linear projection to 256 dimensions → pixel shuffle to 16x16 grid.

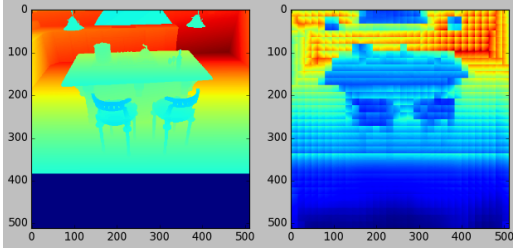


Figure 4: Implicit Neural Representation head.

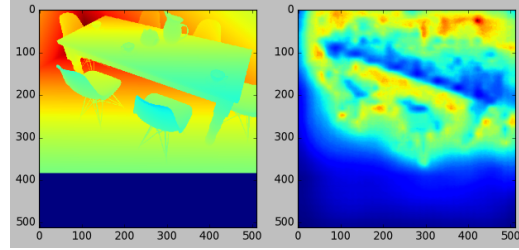


Figure 5: Poorly-implemented DPT head.

The depth loss is given a weight of 3, and all other losses are given a weight of 1. It is worth noting that while the loss *magnitudes* are wildly different, the Smooth L1 loss used means their *gradients* are all 1 (except for the depth loss, whose gradient is 3), and the optimiser only cares about the gradients.

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{point}} + 3\mathcal{L}_{\text{depth}} + \mathcal{L}_{\text{gradient}} + \mathcal{L}_{\text{normal}} + \mathcal{L}_{\text{translation}} + \mathcal{L}_{\text{rotation}}$$

The model is trained for 100 epochs with the AdamW optimiser¹ on a Cosine Annealing with Linear Warmup schedule, with a peak learning rate of 10^{-5} , then 10^{-6} , and so on until proper fine-tuning is achieved. A weight decay of 0.04 is used, alongside a layer-wise learning rate decay of 0.95. Training was performed on 4 H200s.

4.3 Results

A depth head which only takes the final feature map as input significantly underperforms a DPT head (Fig. 2). This is because, as one progresses through more layers, the physical information regarding specific edges and corners, etc. from the earlier layers of the model is easily overwritten by semantic information in later layers. The result is that either the head cannot predict sharp details, or that the model is restricted from overwriting nearly any information from earlier layers. Both of these are undesirable outcomes, which is why a DPT head uses feature maps extracted from four different points within the model.

Directly using a linear layer leads to blocky outputs, because there is zero geometric inductive bias (Fig. 3).

The author additionally experimented with an Implicit Neural Representation-style head, where intra-patch coordinates are concatenated to the feature dimension and passed through an MLP. While this does provide *intra*-patch inductive bias, it lacks *inter*-patch inductive bias (Fig. 4), and uses significantly more VRAM in the process.

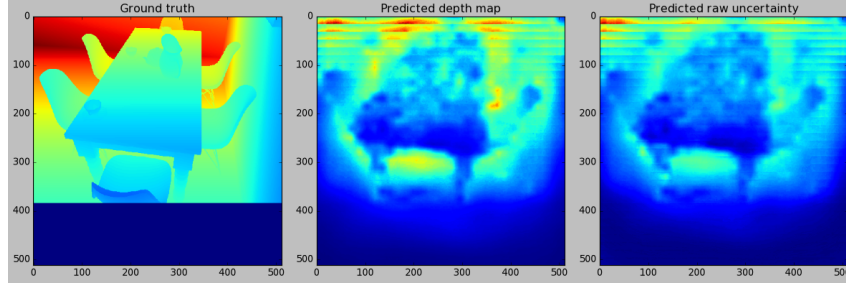
Thus, one must use a DPT head. However, a DPT head implemented poorly will still give terrible results (Fig. 5).

The decision of which layers to pass into the DPT head matters, because the model must accumulate the information required for each layer without overriding anything until that layer has passed. On

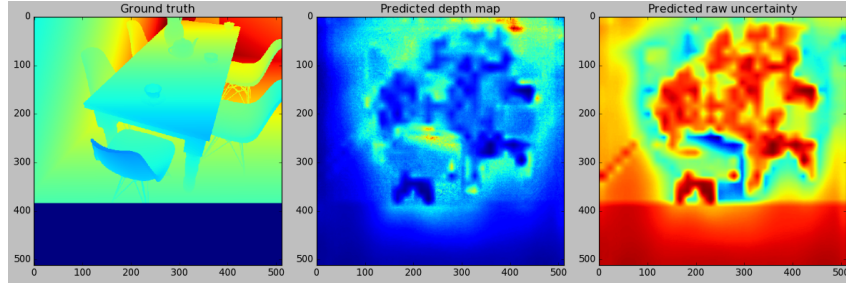
¹The author found that the Muon optimiser gave suboptimal results.

the other hand, choosing layers too early in the pipeline could leave the model not enough capacity to adequately aggregate the information required for a certain layer. The final PP3DR architecture has 16 feature extractor blocks and 18 global/local blocks each; it extracts the fourth and 16th feature extractor layers, and the 9th and 18th local block layers. This means two of the blocks extracted focus entirely on details for the frame itself, and the other two can focus on aggregated global information (e.g. depth uncertainties that can be resolved by looking at other frames).

One might think: if the decision of specifically which intermediate layers are passed to the DPT head is somewhat arbitrary, why not let the model learn individualised weightings for each layer? We can split the combined layers between the feature extractor and the main 3D decoder blocks into four sections and learn per-layer weights for each of these four chunks. In fact, if we’re concerned about the compounding residual connections, we can even weight the *differences* between layers and pass those to each of the four chunks. However, both these options empirically give worse results than simply picking four layers (Fig. 6).



(a) Learned layer weights



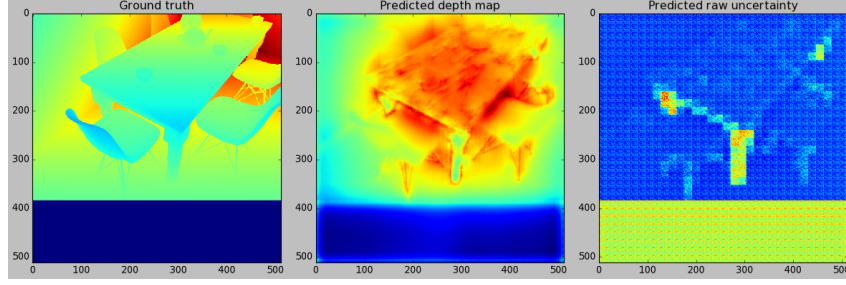
(b) Learned difference weights

Figure 6: Learned weighting.

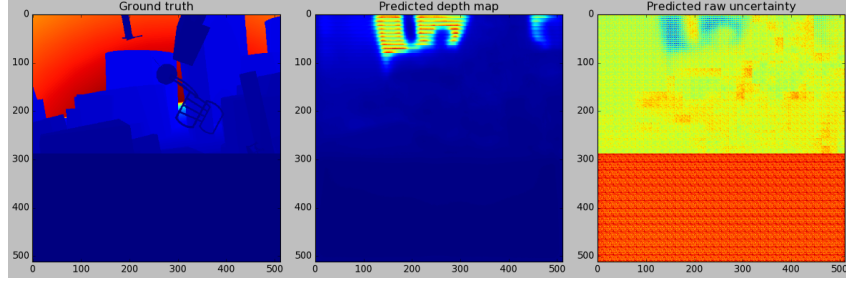
Thus, one has arrived at the final DPT head design (more or less). However, initial results with this head had issues. It’s clear this architecture is capable of resolving great detail, but the actual depths themselves are lacking (Fig. 7a). Also, the predicted uncertainties have severely degraded; this is because this result was obtained with an exp1 aleatoric uncertainty scheme. The FlyingThings3D result is revealing: the vertical banding in the far depths reveals that the normal loss has overrun the depth accuracy (Fig. 7b).

5 Discussion and Future Work

This work is as of yet incomplete, as results competitive with the current state-of-the-art in 3D reconstruction from casual monocular videos were not obtained. By far the most difficult aspect of designing and training a 3D reconstruction model has to do with its dense depth prediction, whose results are heavily influenced by small changes in depth-head architecture and loss design/weighting and which tend to require long training regimes with large quantities of data to converge consistently. VGGT-Omega [24] trained on 128 96GB H100 GPUs for 240K total iterations; the resources available at the University of Maryland Institute for Advanced Computer Studies Vision and Learning Lab allowed training on only 4 140GB H200 GPUs at a time, for at most 36 hours at a time (around 36K iterations).



(a) NRGBD



(b) FlyingThings3D

Figure 7: Initial DPT head results.

Time and resources permitting, the author would like to:

- Further experiment with changes to the model architecture, particularly the DPT head architecture.
- Greatly expand the training curriculum to include many more datasets (such as Co3Dv2 [15] and DL3DV [11]).

However, despite the difficulty encountered, the author firmly believes the improvements elsewhere in the model architecture/parameterisation as well as the promising results of TTT make competitive results from an end-to-end TTT architecture for 3D reconstruction trained from scratch possible in the near future.

6 Acknowledgements

The author would like to thank the University of Maryland Institute for Advanced Computer Studies Vision and Learning Lab for the compute resources used in this work.

References

- [1] Dejan Azinović, Ricardo Martin-Brualla, Dan B Goldman, Matthias Nießner, and Justus Thies. Neural rgb-d surface reconstruction, 2022.
- [2] Daniel J. Butler, Jonas Wulff, Garrett B. Stanley, and Michael J. Black. A naturalistic open source movie for optical flow evaluation. In Andrew Fitzgibbon, Svetlana Lazebnik, Pietro Perona, Yoichi Sato, and Cordelia Schmid, editors, *Computer Vision – ECCV 2012*, pages 611–625, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.
- [3] Xingyu Chen, Yue Chen, Yuliang Xiu, Andreas Geiger, and Anpei Chen. Ttt3r: 3d reconstruction as test-time training, 2026.
- [4] Sven Elflein, Ruilong Li, Sérgio Agostinho, Zan Gojcic, Laura Leal-Taixé, Qunjie Zhou, and Aljosa Osep. Vgg-t³: Offline feed-forward 3d reconstruction at scale, 2026.
- [5] Dongchen Han, Yining Li, Tianyu Li, Zixuan Cao, Ziming Wang, Jun Song, Yu Cheng, Bo Zheng, and Gao Huang. Vit³: Unlocking test-time training in vision, 2026.
- [6] Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, New York, NY, USA, 2 edition, 2003.
- [7] Rasmus Jensen, Anders Dahl, George Vogiatzis, Engin Tola, and Henrik Aanaes. Large Scale Multi-View Stereopsis Evaluation. In *Conference on Computer Vision and Pattern Recognition*, 2014.
- [8] Nikita Karaev, Ignacio Rocco, Benjamin Graham, Natalia Neverova, Andrea Vedaldi, and Christian Rupprecht. Dynamicstereo: Consistent dynamic depth from stereo videos, 2023.
- [9] Wenbin Li, Sajad Saeedi, John McCormac, Ronald Clark, Dimos Tzoumanikas, Qing Ye, Yuzhong Huang, Rui Tang, and Stefan Leutenegger. Interiornet: Mega-scale multi-sensor photo-realistic indoor scenes dataset, 2018.
- [10] Haotong Lin, Sili Chen, Junhao Liew, Donny Y. Chen, Zhenyu Li, Guang Shi, Jiashi Feng, and Bingyi Kang. Depth anything 3: Recovering the visual space from any views, 2025.
- [11] Lu Ling, Yichen Sheng, Zhi Tu, Wentian Zhao, Cheng Xin, Kun Wan, Lantao Yu, Qianyu Guo, Zixun Yu, Yawen Lu, Xuanmao Li, Xingpeng Sun, Rohan Ashok, Aniruddha Mukherjee, Hao Kang, Xiangrui Kong, Gang Hua, Tianyi Zhang, Bedrich Benes, and Aniket Bera. D13dv-10k: A large-scale scene dataset for deep learning-based 3d vision, 2023.
- [12] Junchen Liu, Sven Elflein, Or Litany, Zan Gojcic, and Ruilong Li. Test-time training with kv binding is secretly linear attention, 2026.
- [13] Nikolaus Mayer, Eddy Ilg, Philip Hausser, Philipp Fischer, Daniel Cremers, Alexey Dosovitskiy, and Thomas Brox. A large dataset to train convolutional networks for disparity, optical flow, and scene flow estimation. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, page 4040–4048. IEEE, 2016.
- [14] René Ranftl, Alexey Bochkovskiy, and Vladlen Koltun. Vision transformers for dense prediction, 2021.
- [15] Jeremy Reizenstein, Roman Shapovalov, Philipp Henzler, Luca Sbordone, Patrick Labatut, and David Novotny. Common objects in 3d: Large-scale learning and evaluation of real-life 3d category reconstruction, 2021.
- [16] Johannes L. Schonberger and Jan-Michael Frahm. Structure-from-motion revisited. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2016.
- [17] Thomas Schops, Johannes L. Schonberger, Silvano Galliani, Torsten Sattler, Konrad Schindler, Marc Pollefeys, and Andreas Geiger. A Multi-View Stereo Benchmark with High-Resolution Images and Multi-Camera Videos. In *Conference on Computer Vision and Pattern Recognition*, 2017.
- [18] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, Francisco Massa, Daniel Haziza, Luca Wehrstedt, Jianyuan Wang, Timothée Darcet, Théo Moutakanni, Leonel Sentana, Claire Roberts, Andrea Vedaldi, Jamie Tolan, John Brandt, Camille Couprie, Julien Mairal, Hervé Jégou, Patrick Labatut, and Piotr Bojanowski. Dinov3, 2025.
- [19] Noah Snavely, Steven M. Seitz, and Richard Szeliski. Photo tourism: exploring photo collections in 3d. *ACM Trans. Graph.*, 25(3):835–846, July 2006.

- [20] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2023.
- [21] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to (learn at test time): Rnns with expressive hidden states, 2025.
- [22] Jonathan Tremblay, Moustafa Meshry, Alex Evans, Jan Kautz, Alexander Keller, Sameh Khamis, Thomas Müller, Charles Loop, Nathan Morrical, Koki Nagano, Towaki Takikawa, and Stan Birchfield. Rtmv: A ray-traced multi-view synthetic dataset for novel view synthesis, 2022.
- [23] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. Vggt: Visual geometry grounded transformer, 2025.
- [24] Jianyuan Wang, Minghao Chen, Shangzhan Zhang, Nikita Karaev, Johannes Schönberger, Patrick Labatut, Piotr Bojanowski, David Novotny, Andrea Vedaldi, and Christian Rupprecht. Vggt- ω , 2026.
- [25] Qianqian Wang, Yifei Zhang, Aleksander Holynski, Alexei A. Efros, and Angjoo Kanazawa. Continuous 3d perception model with persistent state, 2025.
- [26] Yifan Wang, Jianjun Zhou, Haoyi Zhu, Wenzheng Chang, Yang Zhou, Zizun Li, Junyi Chen, Jiangmiao Pang, Chunhua Shen, and Tong He. π^3 : Permutation-equivariant visual geometry learning, 2026.
- [27] Junyi Zhang, Charles Herrmann, Junhwa Hur, Chen Sun, Ming-Hsuan Yang, Forrester Cole, Trevor Darrell, and Deqing Sun. Logger: Long-context geometric reconstruction with hybrid memory, 2026.
- [28] Tianyuan Zhang, Sai Bi, Yicong Hong, Kai Zhang, Fujun Luan, Songlin Yang, Kalyan Sunkavalli, William T. Freeman, and Hao Tan. Test-time training done right, 2025.
- [29] Yi Zhou, Connelly Barnes, Jingwan Lu, Jimei Yang, and Hao Li. On the continuity of rotation representations in neural networks, 2020.